SISTEMAS LEGADOS 26/27

Memoria de la Práctica 2



MIERCOLES B 15:00-18:00

GRUPO 03

Alejo Beamonte Andreu (899577)

Izan Celorrio Caballero (900013)

Pablo Redondo Buenacasa (899595)

Índice

1. Acceso y análisis de la aplicación legada..... 2
Figura 1. Menú principal de TAREAS.C..... 2
Figura 2. Creación manual de una tarea general y una tarea específica.....3
Figura 3. Menú de ws3270 navegable por comandos.....3
2. Desarrollo de la capa de acceso en Java..... 4
2.1. Implementación de la comunicación con el sistema legado..... 4
2.2. Problemas detectados y adaptación al comportamiento de TAREAS.C.....4
Figura 4. Ejemplo del comportamiento incorrecto de TAREAS.C al introducir nombres o descripciones con espacios en blanco.....5
Figura 5. Ejemplo de salto de pantalla durante la creación de una tarea específica, que obliga a avanzar una página adicional antes de mostrar el siguiente campo..... 5
Figura 6. Ejemplo de texto residual fusionado en pantalla al consultar listas consecutivas, que obliga a aplicar un filtrado mediante expresiones regulares para extraer la información correcta.....6
3. Desarrollo e integración de la interfaz gráfica..... 6
Figura 7. Interfaz Gráfica de Usuario..... 7
4. Empaquetado, distribución, control de versiones y uso de IA..... 8
4.1. Distribución standalone en Windows y dependencias de tiempo de ejecución..... 8
4.2. Repositorio de control de versiones (Git y GitHub).....8
4.3. Uso de IA.....9
5. Reparto del trabajo..... 9

1. Acceso y análisis de la aplicación legada

Antes de comenzar el desarrollo de la nueva interfaz fue necesario acceder manualmente a la aplicación legada y estudiar su funcionamiento. Para ello se instaló y configuró **wc3270**, estableciendo una conexión TN3270 con el mainframe de la práctica (`155.210.152.51:3270`).

Tras conectarnos a **MUSIC/SP** con la cuenta asignada al grupo, se comprobó el proceso de inicio de sesión. Durante las pruebas se observó que, si existía una sesión anterior abierta con el mismo usuario, MUSIC/SP mostraba el mensaje `Userid is in use`, siendo necesario introducir `OK` para cancelar dicha sesión y continuar.

Una vez dentro del sistema se localizó la aplicación de la práctica. Aunque en el enunciado se hacía referencia a `tasks.c`, en nuestra cuenta el fichero disponible era `TAREAS.C`. Su ejecución desde la línea `Command ===>` inicia el programa mediante Waterloo C y muestra su menú principal:

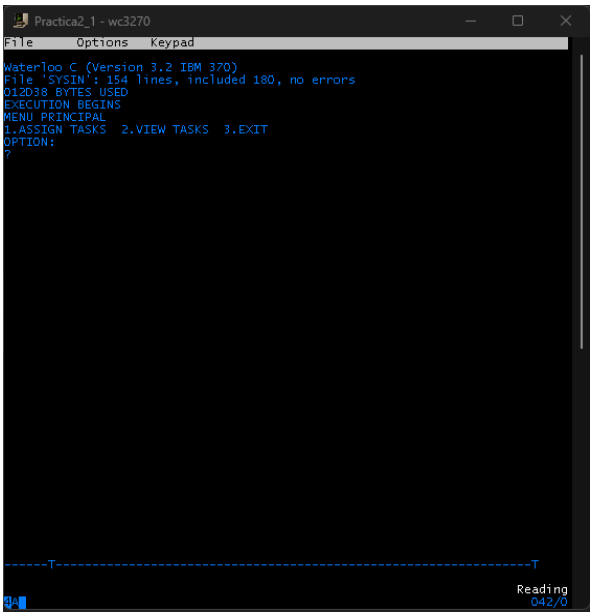


Figura 1. Menú principal de `TAREAS.C`.

A continuación se recorrieron manualmente las distintas opciones para conocer el flujo que posteriormente debía reproducirse desde Java. En `ASSIGN TASKS` se distinguen dos tipos de tareas:

- **General Task:** solicita una fecha en formato `DDMM` y una descripción.
- **Specific Task:** solicita fecha en formato `DDMM`, nombre de la persona y descripción.

Al finalizar correctamente cada operación, la aplicación muestra respectivamente `GENERAL TASK ASSIGNED` o `SPECIFIC TASK ASSIGNED` y regresa al menú de asignación. Desde este submenú, la opción `3. MAIN MENU` permite regresar al menú principal.

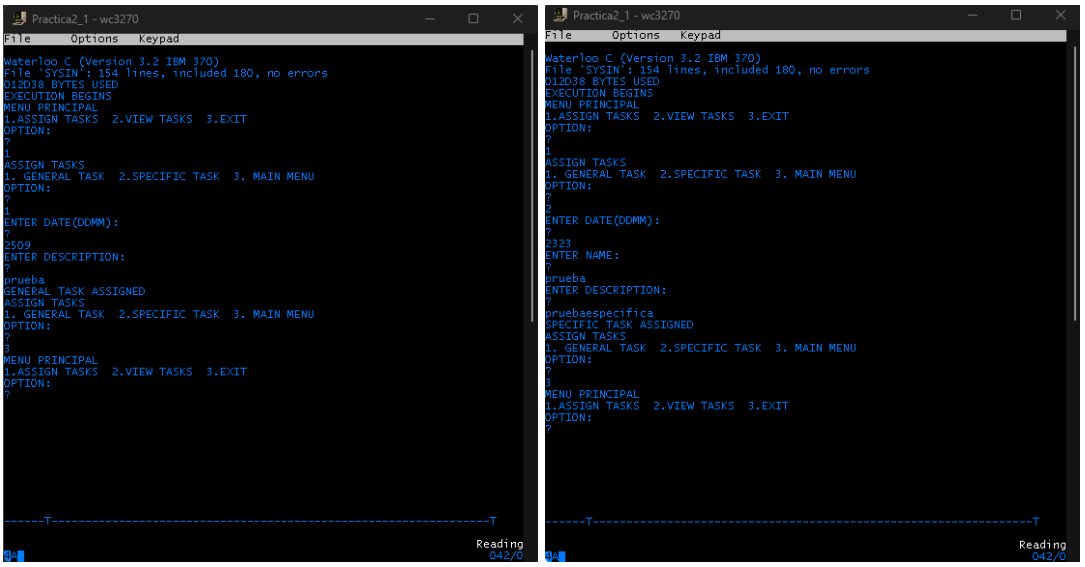
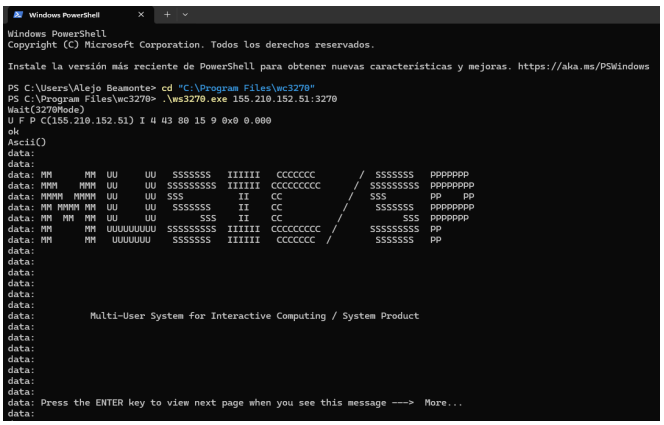


Figura 2. Creación manual de una tarea general y una tarea específica.

También se comprobó la opción `VIEW TASKS`, que permite consultar por separado tareas generales y específicas. Esta funcionalidad resulta especialmente relevante, ya que posteriormente será necesario obtener el contenido mostrado por el terminal y transformarlo en información utilizable por la interfaz gráfica. Finalmente se estudió la salida de la aplicación. Desde el menú principal, la opción `3. EXIT` finaliza `TAREAS.C` y muestra una pantalla del tipo `Press enter to continue...`, siendo necesario pulsar nuevamente `Enter` para regresar a MUSIC/SP.

Se realizaron además pruebas mediante **ws3270**, la versión sin interfaz gráfica del emulador. Se comprobó el funcionamiento de comandos:



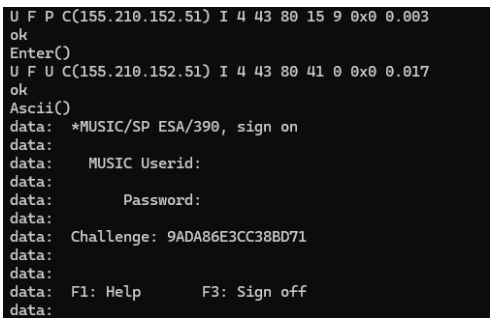


Figura 3. Menú de ws3270 navegable por comandos

Esto permitió verificar que todas las operaciones realizadas manualmente podían automatizarse mediante comandos desde un programa externo.

Con estas pruebas quedó definido el flujo completo de interacción con la aplicación legada y se confirmó la viabilidad de utilizar `ws3270` mediante comandos.

2. Desarrollo de la capa de acceso en Java

2.1. Implementación de la comunicación con el sistema legado

Una vez analizado manualmente el funcionamiento de `TAREAS.C`, se desarrolló una capa en Java encargada de automatizar la comunicación con el mainframe mediante `ws3270`.

La clase `Ws3270Client` centraliza esta comunicación, permitiendo establecer la conexión con el mainframe, iniciar sesión en MUSIC/SP, ejecutar la aplicación legada y enviar las acciones necesarias para interactuar con sus distintos menús.

Sobre esta base se implementaron todas las funcionalidades necesarias de la aplicación, incluyendo la creación y consulta de tareas generales y específicas, la navegación entre menús y la finalización correcta de la sesión. Además, se añadieron comprobaciones para detectar errores de comunicación y verificar que cada operación alcanza el estado esperado antes de continuar.

De esta forma, la lógica necesaria para interactuar con el sistema legado queda encapsulada en una única capa, permitiendo que el resto de la aplicación utilice estas operaciones sin depender directamente de los comandos propios del terminal 3270.

2.2. Problemas detectados y adaptación al comportamiento de TAREAS.C

Durante el desarrollo se comprobó que la principal dificultad no estaba en la conexión con el mainframe, sino en determinados comportamientos de la propia aplicación legada `TAREAS.C`, que no siempre procesa las entradas de forma predecible.

El primer problema detectado apareció al introducir **nombres o descripciones con espacios en blanco** al crear tareas. En estas situaciones, la aplicación no trataba correctamente el texto introducido y podía pasar directamente a otra petición de datos o dejar de volver al menú correspondiente. En otras palabras, algo tan normal como escribir una descripción formada por varias palabras hacía que el flujo de la aplicación se rompiera. Sin embargo, al introducir valores sin espacios, el comportamiento era correcto. Por este motivo, se decidió adaptar la solución validando estos campos y limitándolos a una única palabra sin espacios, evitando así errores en la ejecución.

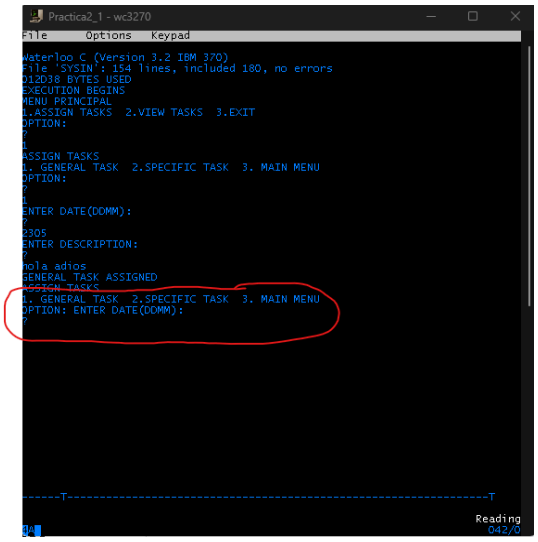


Figura 4. Ejemplo del comportamiento incorrecto de `TAREAS.C` al introducir nombres o descripciones con espacios en blanco.

El segundo problema estaba relacionado con la forma en que la aplicación muestra la información en pantalla. En algunos pasos, especialmente durante la creación de tareas específicas, la pantalla podía llenarse y pasar a una nueva página antes de mostrar el siguiente campo. Esto hacía que, después de introducir un dato, no siempre apareciera inmediatamente la siguiente petición, sino que fuera necesario pulsar `Enter` una vez más para continuar y pasar a la siguiente página. Para resolverlo, se implementó una comprobación adicional que verifica si el mensaje esperado aparece en pantalla y, si no es así, envía automáticamente un `Enter` extra para avanzar de página y continuar con normalidad. De este modo, el código no solo reproduce las acciones básicas del usuario, sino que también tiene en cuenta estas particularidades de `TAREAS.C`, haciendo la automatización más estable y reutilizable.

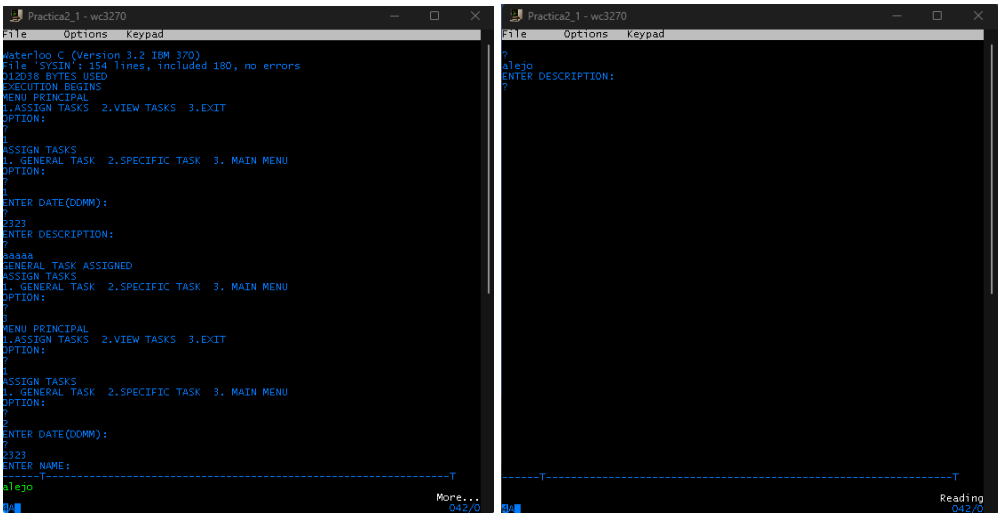


Figura 5. Ejemplo de salto de pantalla durante la creación de una tarea específica, que obliga a avanzar una página adicional antes de mostrar el siguiente campo

Adicionalmente a los problemas detectados durante la creación de tareas, la fase de lectura y extracción (VIEW TASKS) presentó nuevos desafíos:

En primer lugar, se observó que el sistema del mainframe no siempre "limpia" la pantalla al mostrar nuevas consultas, sino que sobrescribe el nuevo texto. Esto provocaba que, al listar tareas de un tipo inmediatamente después de otro, aparecieran restos de texto fusionados pertenecientes a consultas anteriores. Para evitar capturar esta "basura" visual y asignar tareas erróneas, se implementó un filtrado mediante expresiones regulares. Entonces se planteó capturar de forma exclusiva aquellas líneas que cumplan un patrón exacto (^TASK \d+: TIPO.*), garantizando una extracción de datos limpia.

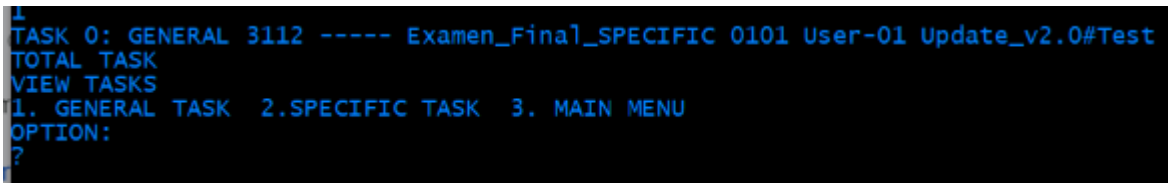


Figura 6. Ejemplo de texto residual fusionado en pantalla al consultar listas consecutivas, que obliga a aplicar un filtrado mediante expresiones regulares para extraer la información correcta.

En segundo lugar, la lectura de listados de tareas extensos provocaba una interrupción en el flujo del terminal. Al llenarse las 24 líneas de la pantalla, el sistema operativo mostraba el texto More... en la esquina inferior derecha, bloqueando la ejecución hasta recibir interacción. Este obstáculo se resolvió mediante la implementación de un bucle de paginación dinámica: el programa evalúa cíclicamente la pantalla capturada y, si detecta dicho indicador, envía automáticamente un comando Enter adicional para avanzar de página, repitiendo el proceso hasta recuperar el listado completo.

Finalmente, mediante pruebas de estrés sobre la aplicación legada, se descubrió una grave vulnerabilidad de desbordamiento de memoria (Buffer Overflow). Al intentar crear una tarea con una descripción inusualmente larga (por ejemplo, 70 caracteres), se comprobó que el programa TAREAS.C carece de límite de tamaño en sus entradas. El exceso de caracteres invadía el espacio de memoria reservado para otras tareas, sobrescribiendo sus valores y rompiendo el fichero de datos interno del servidor. Para proteger al mainframe de esta vulnerabilidad, se amplió la barrera de seguridad de esta capa, estableciendo que ninguna cadena de texto (nombres o descripciones) enviada al servidor pueda superar un límite estricto de 20 caracteres.

3. Desarrollo e integración de la interfaz gráfica

Se ha desarrollado una interfaz gráfica de ventanas utilizando la librería estándar Java Swing, adoptando el aspecto visual nativo del sistema operativo (System Look and Feel) para ofrecer una experiencia de usuario moderna, limpia y profesional. El diseño sigue fielmente las directrices y el modelo de referencia sugerido en el guion de la práctica, desacoplando por completo al usuario del arcaico entorno de terminal de caracteres e integrando: • Un panel superior con formulario interactivo: permite introducir directamente la fecha, la descripción y el destinatario de la tarea, gestionando de forma dinámica la activación del campo de nombre según se seleccione una tarea de tipo General o Específica. • Un panel central con tabla de datos estructurada (JTable): presenta el listado

consolidado de tareas recuperadas del mainframe organizadas en columnas claras (Nº, Tipo, Fecha en formato DD/MM, Asignado a y Descripción) con soporte para desplazamiento vertical (JScrollPane). • Un panel lateral de control: botones accesibles para iniciar la conexión, recargar las tareas bajo demanda y finalizar la sesión. • Una barra de estado inferior (Status Bar): informa en todo momento del estado de la conexión con MUSIC/SP y del número total de tareas sincronizadas en memoria.

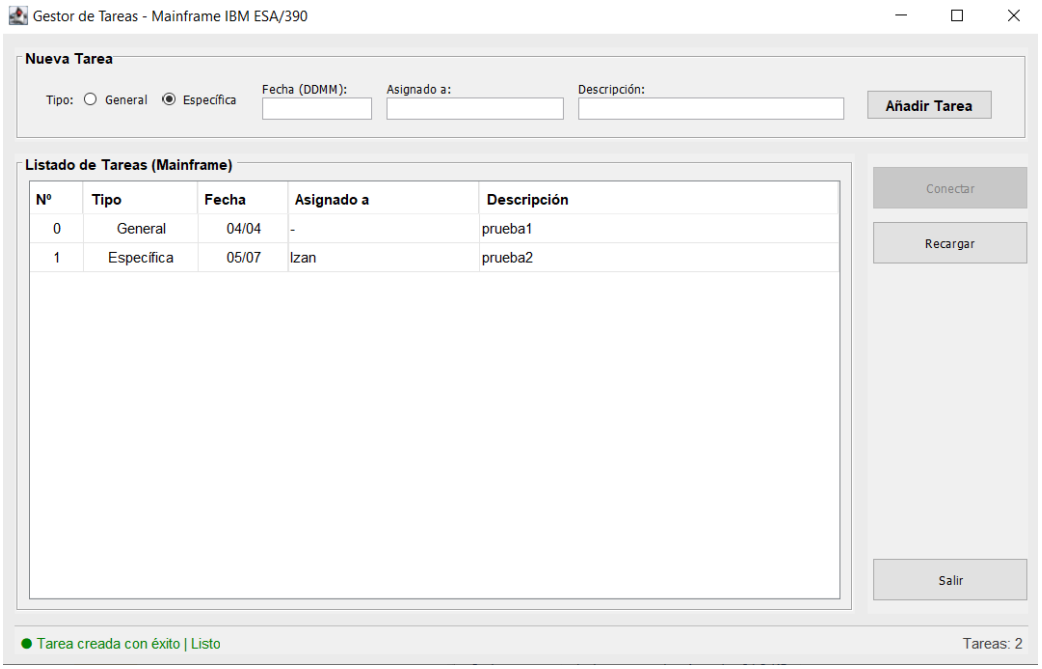


Figura 7. Interfaz Gráfica de Usuario

Para garantizar la estabilidad tanto de la aplicación local como del servidor remoto, la interfaz implementa un sistema de control de estado con las siguientes características:

- Validación estricta en el formulario: Antes de delegar el trabajo a la capa de comunicación, el formulario valida en tiempo real la longitud (máximo 20 caracteres), el formato de fecha (DDMM) y la ausencia de espacios en blanco en los parámetros introducidos, protegiendo al mainframe de las vulnerabilidades documentadas en el apartado 2 sin interrumpir el flujo visual de la aplicación.
- Procesamiento asíncrono: Las invocaciones al emulador ws3270 se ejecutan en hilos secundarios. Esto evita la congelación de la ventana principal durante los retardos inherentes a las comunicaciones de red.
- Bloqueo de concurrencia: Mientras la capa Java interactúa con los menús del sistema legado, la interfaz deshabilita por completo todos sus controles. Esto impide que el usuario envíe múltiples comandos superpuestos que desincronicen el flujo de las pantallas.
- Desconexión segura: Se ha interceptado el evento de cierre de la ventana para evitar cortes abruptos. Al intentar salir de la aplicación, el sistema envía un comando de cierre al servidor y aplica una pausa obligatoria de 1,5 segundos antes de finalizar el proceso Java, cumpliendo así el requisito de evitar bloqueos en el SO MUSIC/SP.

4. Empaquetado, distribución, control de versiones y uso de IA

4.1. Distribución standalone en Windows y dependencias de tiempo de ejecución

Con el fin de facilitar la corrección y permitir la ejecución inmediata de la aplicación haciendo doble clic sobre GestorTareas.exe en cualquier equipo con Windows sin exigir la instalación previa ni configuración del entorno Java ni del emulador en la consola, se investigó la posibilidad de compilar un binario 100% estático y monolítico.

Sin embargo, tras analizar la arquitectura de la solución basada en la Máquina Virtual de Java (JVM) y el protocolo de emulación 3270, se constató que no es técnicamente viable absorber todo el entorno dentro de un único ejecutable sin dependencias externas. Por un lado, las clases compiladas de Java residen en formato bytecode dentro de un archivo JAR («GestorTareas.jar»), el cual requiere la presencia de un entorno de ejecución Java (JRE) y depende habitualmente de la invocación por línea de comandos o de la asociación de ficheros en el sistema operativo. Por otro lado, la interacción con el mainframe IBM ESA/390 depende estrictamente de un ejecutable nativo de emulación 3270 («ws3270.exe»), cuya naturaleza como binario Win32 independiente impide embeberlo directamente en el espacio de memoria del proceso Java y cuya ruta en el código fuente no puede fijarse a directorios absolutos del sistema (como C:\Program Files\wc3270\ws3270.exe) sin romper la portabilidad en equipos ajenos.

Por este motivo, se adoptó la siguiente solución de distribución:

1. Se utilizó la herramienta Launch4j para encapsular el archivo JAR ejecutable («GestorTareas.jar») —que contiene el manifiesto correspondiente señalando a la clase principal de la interfaz gráfica («unizar.legados.AppGUI»)— dentro de un binario ejecutable nativo de Windows («GestorTareas.exe»). Este wrapper Win32 localiza automáticamente el entorno de ejecución Java (JRE/JDK) instalado en el sistema operativo (con versión mínima 1.8) o emite un aviso visual informativo si no está presente, permitiendo el arranque directo por doble clic sin requerir consolas ni scripts intermedios de tipo fichero por lotes (.bat).

2. Se implementó en la capa de acceso Java un mecanismo de resolución dinámica de rutas que busca de manera prioritaria el emulador «ws3270.exe» en el propio directorio de trabajo local donde reside la aplicación. De esta forma, se empaqueta en el mismo directorio de entrega el ejecutable «GestorTareas.exe» (y «GestorTareas.jar») junto al binario portable «ws3270.exe». De esta forma, el cargador de Windows y el proceso de la aplicación resuelven las dependencias en la carpeta local, logrando una ejecución directa por doble clic completamente autónoma y funcional.

4.2. Repositorio de control de versiones (Git y GitHub)

Todo el ciclo de vida del proyecto, el historial de cambios, el código fuente, los programas auxiliares y la documentación técnica han sido gestionados mediante control de versiones

en Git y se encuentran sincronizados y accesibles en el repositorio oficial de GitHub del grupo: https://github.com/izancelorrio/Sistemas_legados.git

4.3. Uso de IA

Durante el desarrollo de la práctica se ha recurrido de forma puntual al uso de herramientas de Inteligencia Artificial generativa exclusivamente como soporte auxiliar. Su utilización se ha circunscrito a labores secundarias de asistencia en la redacción, maquetación y estilo de la memoria técnica, así como a consultas puntuales de sintaxis sobre componentes de la API de Java y utilidades de empaquetado. Toda la carga sustancial y técnica del proyecto, el análisis inverso de la aplicación legada **`TAREAS.C`**, el diagnóstico y resolución de las anomalías de sincronización del terminal 3270 (saltos de página, textos residuales y mitigación de desbordamientos de memoria), el desarrollo íntegro de la capa de comunicación y el diseño y lógica de la interfaz gráfica, ha sido concebida, implementada, depurada y verificada en su totalidad por los miembros del grupo.

5. Reparto del trabajo

La práctica se ha desarrollado de forma colaborativa entre los tres miembros del grupo, distribuyendo las tareas principales por bloques funcionales para reducir dependencias y facilitar la integración posterior.

Alumno	Tareas	Horas aprox.
Alejo Beamonte Andreu (899577)	- Análisis inicial de la aplicación legada `TAREAS.C` y comprobación manual de su funcionamiento mediante `wc3270` y `ws3270`. - Desarrollo de la capa de acceso en Java para la conexión con el mainframe, inicio de sesión en MUSIC/SP y ejecución de `TAREAS.C`. - Implementación de las funcionalidades de creación de tareas generales y específicas, así como de la navegación entre menús. - Identificación y resolución de problemas relacionados con los espacios en blanco y los saltos de página del terminal. - Redacción de los apartados correspondientes al acceso al sistema legado y al desarrollo de la capa Java en la memoria.	**7 h**

Izan Celorrio Caballero (900013)	- Configuración del empaquetado y distribución portátil de la aplicación (creación del JAR ejecutable e integración mediante Launch4j para generar el ejecutable .exe por doble clic). - Adaptación de la capa de comunicación Ws3270Client para la resolución dinámica y localización local del binario ws3270.exe. - Detección, diagnóstico y resolución de los fallos de sincronización con el terminal 3270 (mecanismos de reenvío en línea de entrada, mitigación del salto de pantalla/rollover y prevención de envíos prematuros). - Gestión del repositorio Git, limpieza de ficheros temporales y configuración del archivo .gitignore. - Redacción del apartado de empaquetado, distribución y control de versiones en la memoria técnica.	**8 h**
Pablo Redondo Buenacasa (899595)	- Implementación de la funcionalidad de visualización y extracción de tareas, además de la interfaz gráfica. - Pruebas para descubrir vulnerabilidades en casos límites y la posterior resolución de los mismos (paginación, buffer overflow, uso de expresiones regulares para extraer correctamente el texto…). - Redacción y actualización de la memoria en sus apartados correspondientes al desarrollo Java y la interfaz gráfica	**8 h**